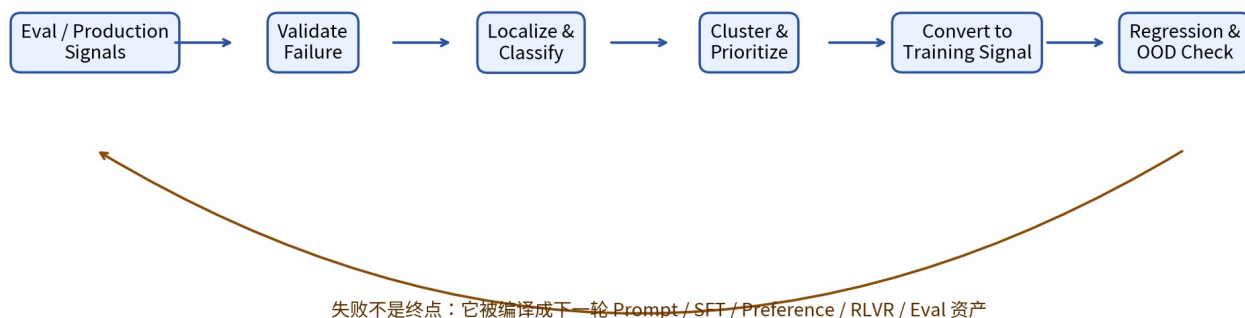


LLM 后训练之 Failure Mining

失败挖掘：从评测错误到下一轮训练信号的工程闭环

Deep Research & Engineering Handbook

更新：2026-08-13



核心判断：Failure Mining 不是“收集 bad cases”，而是把真实失败验证、定位、聚类、优先级排序，并编译成可执行的 Prompt / Data / Reward / Eval / Product 修复任务。

目录与研究结论

#	章节
01	Failure Mining 的第一性定义
02	为什么 Evaluation 之后必须有 Failure Mining
03	Failure 的输入源与最小分析单位
04	先做 Failure Validation：模型错还是 Eval 错
05	Failure Taxonomy：建立可累计语言
06	Trajectory Root-Cause：定位关键失败步骤
07	Cluster / Slice / Near-Miss Mining
08	自动化 Failure Discovery：Petri、Bloom、Red Team
09	Failure → Training Signal 的 Stage Router
10	Failure Priority 与预算分配
11	SFT / Preference / RLVR 的不同修复方式
12	Agent / Coding / Safety 三个案例
13	指标、数据契约与生产架构
14	18 类常见失败模式
15	成熟度模型与实施 Cookbook
16	核心论文与官方资料

一句话总纲：Eval 负责“暴露差距”，Failure Mining 负责“解释差距并决定下一步”。没有 Failure Mining，后训练容易退化成对 leaderboard 的局部打补丁。

1. Failure Mining 的第一性定义

在 LLM 后训练中，Failure Mining 可以定义为：

Observed Failure → Validated Failure → Root Cause → Failure Cluster → Training / System Intervention

它不是单纯的错误日志分析，也不是只筛选“答错的样本”。真正的对象是：为什么模型在某一类状态下失败、失败是否可复现、哪些相邻条件会触发同类失败、哪种干预最可能产生可泛化的修复。

概念	核心问题	产物
Error Logging	发生了什么？	raw logs / failed trials
Error Analysis	为什么失败？	taxonomy / hypothesis
Hard Example Mining	哪些样本最值得学？	high-value prompt pool
Failure Mining	失败如何形成可执行闭环？	cluster + root cause + route + regression set
Red Teaming	还能主动发现什么新失败？	adversarial cases / novel attack families

Know-Why：训练算法只会优化给它的信号。Failure Mining 的价值在于把模糊的“模型不行”转成有边界、可验证、可训练的 deficiency。

2. 为什么 Evaluation 之后必须有 Failure Mining

OpenAI 2025 的 eval 实践把循环概括为 Specify → Measure → Improve，并明确建议先人工审阅约 50-100 个输出形成 error taxonomy，再持续把新错误加入分析与数据飞轮。[1] Anthropic 的 Agent Eval 实践则强调必须阅读 transcript：单个失败分数无法区分 agent 真错、grader 错、harness 限制或任务歧义。[2]

Evaluation 只告诉你	Failure Mining 继续回答
总体正确率 72%	剩余 28% 是哪几类失败？
Agent task failed	第一处关键错误在哪一步？
Judge score = 2/10	judge 是否可靠？ rubric 是否歧义？
某 slice 回归 -5pp	是数据分布变化、policy drift 还是工具变更？
RL reward 下降	任务变难、rollout 退化还是 verifier 被改变？
用户投诉	是单例、系统性模式还是产品规格冲突？

- 把平均分转换为 failure distribution。
- 把“例子”转换为 failure family。
- 把 failure family 转换为训练/产品 intervention。
- 把修复后的例子重新加入 private regression / OOD eval，防止回归。

3. Failure 的输入源与最小分析单位

输入源	最适合发现	风险/偏差
Offline Eval	已知能力缺口、稳定回归	容易 benchmark-specific
Private Regression	历史已修复问题	覆盖范围有限
Production Logs	真实分布、真实成本	隐私/抽样偏差
User Feedback	高影响 UX failure	噪声大、选择性反馈
Agent Trajectory	中间步骤、工具/恢复问题	长轨迹分析成本高
Judge Disagreement	主观/边界样本	judge 共同偏差
Red Team / Auditor	罕见、对抗、未知失败	现实性与代表性风险
RL Rollouts	learnable frontier / zero-gradient groups	受 sampling policy 强烈影响

3.1 最小分析单位不是固定的

粒度	适用场景	例子
Sample	单轮 QA	知识错误、格式错
Turn	对话	用户纠正后仍坚持错误
Step	Agent/Reasoning	错误工具选择、错误推理分支
Trajectory	长程 Agent	早期错误传播导致最终失败
Slice	统计模式	长上下文 + tool use 同时退化
Cluster	语义族	所有“隐含时区”相关失败

4. Failure Validation：先判断“谁错了”

最危险的 Failure Mining 错误：把 grader / task / environment 的缺陷当成 model failure，然后把错误标签重新训练进模型。

OpenAI 2026 对 SWE-Bench Pro 的审计通过自动 datapoint analysis 和人类标注发现，约 30% 的任务存在破坏性问题，主要包括过度严格测试、欠规格 prompt、测试覆盖不足和误导性 prompt。[3] Anthropic 2026 也报告 CORE-Bench 中因 rigid grading、歧义任务和不可复现随机性等问题，修正后某模型的分数从 42% 提升到 95%。[2]

验证问题	如果答案为 Yes	动作
存在多个合理解但 grader 只接受一种？	Eval bug	修 grader，不训练模型
prompt 是否缺失 hidden test 强制的要求？	Spec bug	修 task spec / 移出训练
环境是否随机或外部状态不可复现？	Environment bug	固定 seed / snapshot
harness 是否不给模型完成任务所需权限？	Harness bug	修系统接口
失败是否能在重跑中稳定复现？	Model candidate failure	进入 root-cause
不同 graders 是否一致？	高置信 failure	进入 cluster / priority

4.1 Failure Confidence

建议为每个 failure 保存诊断置信度，而不是 binary label：

$$C_failure = Reproducibility \times GraderAgreement \times SpecClarity \times EnvironmentStability$$

置信度低的 case 应进入 human / expert review queue，而不是直接进入训练。

5. Failure Taxonomy：建立可累计的共同语言

规格层	目标/任务定义错误	每个 failure 还要附加： • Severity 严重度 • Frequency 频率 • Reproducibility 可复现性 • Confidence 诊断置信度 • Novelty 新颖性 • Trainability 可训练性 • Product impact 产品影响 • Intervention route 修复路线
评测层	grader / hidden test / rubric 错误	
输入层	prompt / context / retrieval 问题	
策略层	reasoning / planning / instruction-following	
工具层	tool selection / args / state	
环境层	nondeterminism / permission / external state	
恢复层	retry / correction / stop failure	

Failure taxonomy 的目的不是写一份漂亮的错误分类文档，而是让不同模型版本、不同数据阶段、不同团队都能用同一种语言累计统计。

一级维度	二级示例	训练价值
Specification	目标冲突 / 欠规格 / 不可满足	通常不是模型训练问题
Instruction	漏约束 / 优先级错 / 格式错	SFT / Preference
Reasoning	错误假设 / 计划 / 计算 / 搜索	Reasoning SFT / RLVR
Knowledge	事实缺失 / 时效性 / 检索失败	RAG / data / tool
Tool	选错工具 / 参数错 / 解析错	Tool SFT / Agent RL
State	忘记先前状态 / 环境同步错	memory / harness / trajectory train
Recovery	失败后不重试 / 重试无策略	correction trajectory / RL
Behavior	sycophancy / overrefusal / deception	preference / safety data
Evaluator	judge bias / test bug / leakage	修 Eval，不修 policy

Know-How: taxonomy 应允许“多标签”。一个 Agent failure 可能同时是 planning error + tool argument error + recovery failure；强行单标签会丢失因果链。

6. Trajectory Root-Cause：从最终失败定位关键步骤

长程 Agent 的最大难题是 error propagation：最终 task failure 只能说明“整条轨迹没完成”，不能说明哪一步应该被训练。2026 的 AgentRx 手工标注了 failed trajectories 的关键失败步骤与跨域 taxonomy；AgentEval 则用 DAG 依赖关系来追踪错误传播，并报告相对 end-to-end eval 更高的 failure detection recall 与 root-cause accuracy。[4][5]

步骤	诊断问题	产物
1. Trace normalization	把 tool calls / observations / state change 结构化	typed trajectory
2. Constraint extraction	每一步应该满足什么？	step assertions
3. First divergence	第一次从可成功路径偏离在哪？	critical step
4. Propagation graph	后续错误是独立还是由上游导致？	dependency DAG
5. Counterfactual check	若替换该步，任务能否恢复？	causal confidence
6. Failure label	归到 taxonomy + severity	root-cause record

关键原则：不要把因上游错误造成的所有下游动作都当成独立负样本。否则会重复惩罚同一个 root cause。

7. Cluster / Slice / Near-Miss Mining

7.1 从 bad case 到 failure family

- 对 failure prompt / trace 做 embedding + metadata clustering。
- 同时保留 taxonomy 标签，避免纯语义聚类把不同 root cause 混在一起。
- 按模型版本、domain、difficulty、context length、tool、language、user segment 做切片。
- 寻找“失败率异常高”的交叉 slice，而不是只看全局平均。

7.2 Near-Miss 比纯失败更有训练价值

完全不可解的样本可能提供极低学习信号；完全会做的样本也几乎没有边际价值。Failure Mining 应寻找当前 policy 的 learnable frontier。

High-value zone $\approx$ medium success probability + high impact + reliable verifier

Case 类型	典型特征	优先级
Always pass	$p \approx 1$	用于 regression，低训练权重
Near-miss	$0.2 < p < 0.8$	最高：有可学习梯度
Always fail	$p \approx 0$	先查是否超能力/规格问题
Flaky	跨 trial 大幅波动	先查 stochasticity / harness
Judge-disputed	judge 分歧高	先人工/专家校准

8. 自动化 Failure Discovery：从被动日志到主动找错

8.1 Petri：探索未知行为

Anthropic 的 Petri 用 auditor agent 在多轮模拟环境中主动测试目标模型，并用 judge 对 transcript 打分，适合开放式探索“还不知道有哪些 failure”。2026 的 Petri 2.0 又增加 realism classifier 来降低 eval-awareness。[6][7]

8.2 Bloom：把已知 failure 变成可测分布

Bloom 与 Petri 的角色互补：Petri 更适合发现；Bloom 输入明确行为描述后自动生成大量场景、rollout 与 judgment，用于测量 failure 的频率、严重度与多样性。[8]

8.3 A3：从一个 failure 自动扩成训练闭环

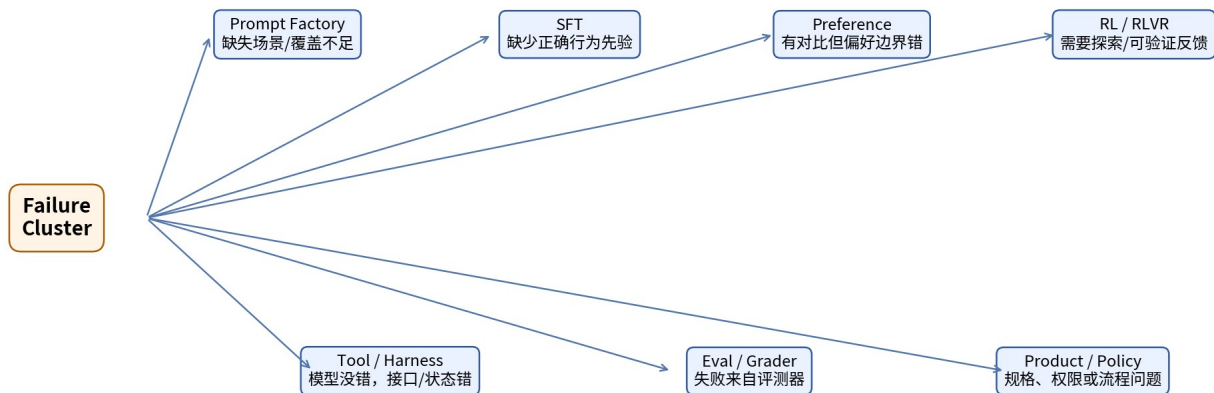
A3 是 Failure Mining → Data → Finetune 的代表性公开案例：从一个不良行为样例开始形成 hypothesis，生成 harmful/benign query，按 hypothesis 做 train/validation/OOD split，再依据当前模型在哪些 hypothesis 上失败来动态重加权训练数据。[9]

8.4 GPT-Red：对抗式 Failure Mining

OpenAI 2026 的 GPT-Red 将 prompt-injection failure discovery 训练成 self-play red-team policy，并把发现的攻击用于 production model 的 adversarial training；公开结果显示这种“攻击者随防守者共同升级”的方式可以形成 robustness self-improvement flywheel。[10]

工具/范式	发现模式	最适合
Passive Logs	真实发生过的失败	产品分布
Petri / Auditor	未知/稀有行为探索	alignment / agent
Bloom	已知行为的系统测量	frequency / severity
A3	单一 failure 扩成数据并修复	targeted finetuning
GPT-Red / Red Team RL	对抗性漏洞主动搜索	robustness / security

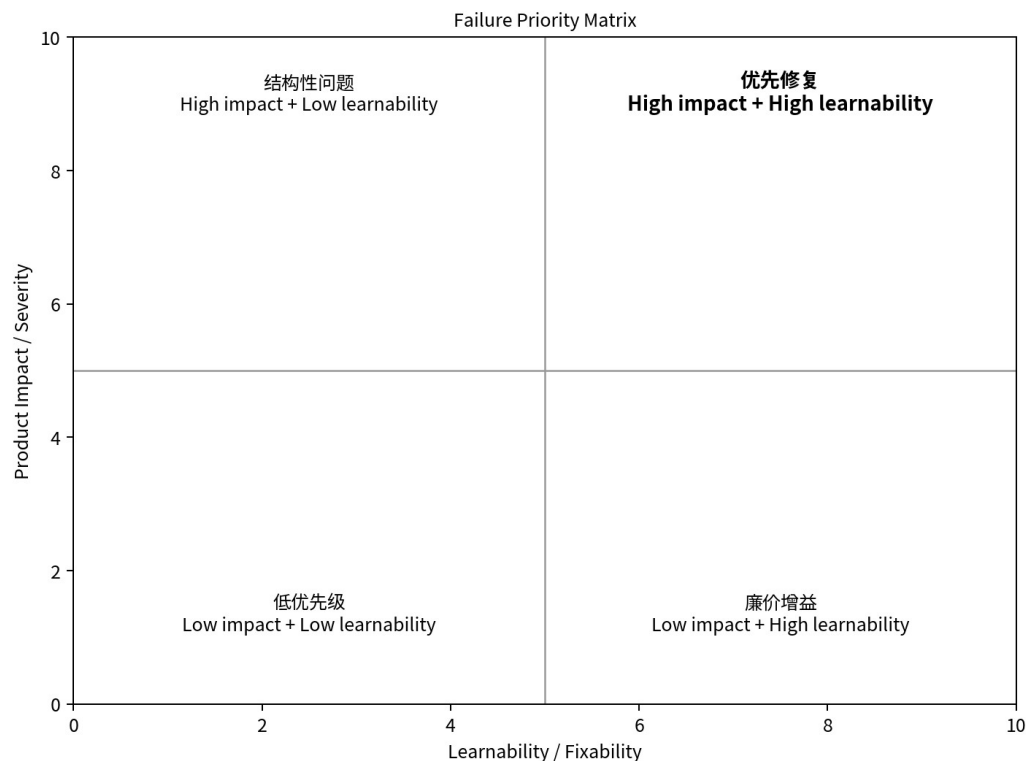
9. Failure → Training Signal : Stage Router



Failure 类型	优先路由	为什么
缺少任务覆盖	Prompt Factory / Synthetic	先制造正确场景
基本行为不会	SFT	需要直接示范
A/B 边界错	Preference Optimization	需要相对偏好
存在可验证 outcome、需要探索	RLVR	reward 可以自动给
长程步骤错误	Trajectory SFT / Agent RL	训练状态-动作序列
事实/外部状态缺失	RAG / Tool / Product	参数训练未必是正确解
Grader 错	Eval	不能用训练掩盖评测错误
系统权限/工具协议错	Harness	修接口比训模型更直接

Failure Mining 的核心不是“多造数据”，而是 routing：同一个最终失败，可能应该修模型、修数据、修 reward、修 evaluator、修 tool schema，甚至修产品目标。

10. Failure Priority ：有限预算下先修什么



$Priority(f) = Impact \times Frequency \times Confidence \times Trainability \times Novelty / Cost$

因素	说明	常见错误
Impact	对用户/任务/安全的真实影响	只按 benchmark 权重
Frequency	真实或目标分布发生率	被高频简单 case 淹没
Confidence	是真 failure 的把握	忽略 grader bug
Trainability	当前模型有无可学习信号	反复训练不可解任务
Novelty	是否已被训练/回归覆盖	重复收同类 bad cases
Cost	生成、标注、验证、训练成本	只算训练 tokens

10.1 Severity 与 Frequency 不应相互替代

高频低影响问题与低频高影响问题应分别建立 budget。成熟系统通常需要 capability queue、product queue、safety queue、regression queue，而不是把所有 failure 放入一个总榜。

11. 不同训练阶段需要不同 Failure-to-Data 编译器

目标阶段	Failure 变成什么数据	关键过滤
SFT	prompt + corrected ideal response	正确性、覆盖、避免只记答案
DPO / IPO	same prompt + chosen/rejected	pair 差异必须指向目标行为
RM / Judge	hard pair / ambiguous pair	防 label noise 与 style shortcut
RLVR	prompt + verifier + rollout group	必须有非零学习信号
Distillation	teacher successful trajectory	student learnability + verifier
Agent SFT	critical step 前后的修复 trajectory	避免把下游传播错当独立错

11.1 Correction Data：别只给“正确答案”

SCoRe 的结果说明，离线 self-correction traces 的 SFT 容易受分布错配和行为坍塌影响；其方法转向在模型自身分布上做多轮 online RL。这提示 Failure Mining 生成 correction data 时，应尽量覆盖“当前 policy 实际会犯的错”，而不是只用 teacher 理想化的错误轨迹。[11]

数据形式	训练信号
Bad → Good	学最终正确行为
Bad + Critique → Good	学检测与修正
Near-miss A vs Better B	学边界偏好
Failure trajectory → repaired trajectory	学恢复策略
Failure + benign counterpart	降低过度修复/false positive

12. 三个端到端案例

12.1 Coding Agent：测试失败 $\Rightarrow$ 模型修复失败

1. 收集 failed repo tasks 与完整 tool trace。
2. 先审计 task spec / hidden tests / environment；剔除 broken tasks。
3. 找到第一处关键偏离：检索错误、定位错误、patch 错、test/recovery 错。
4. 把同类 failure 聚成 repo-understanding / diagnosis / edit / test / recovery 五类。
5. 针对 root cause 生成对比数据或修复 trajectory。
6. 保留原 failure + OOD 变体进入 private regression。

12.2 Safety / Robustness：一个 jailbreak 扩成 failure family

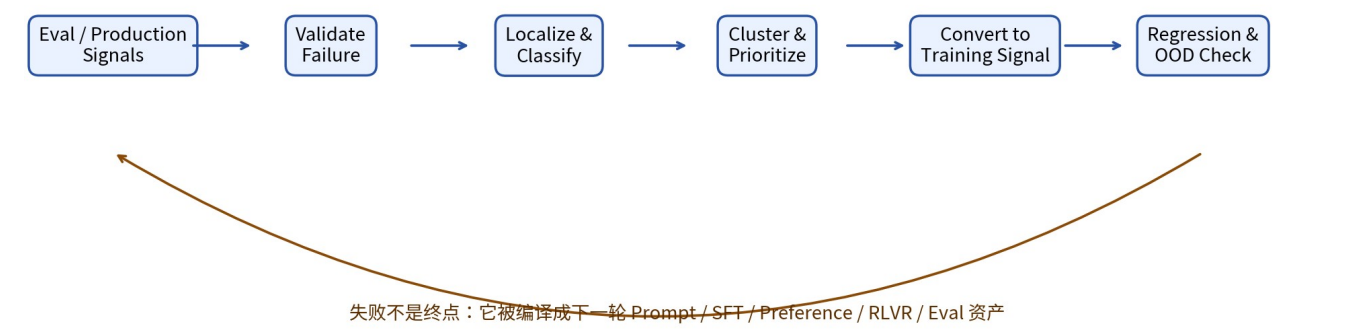
7. 以真实或 audit 发现的失败为 Ex0。
8. 提炼触发机制 hypothesis，而不是只改写原 prompt。
9. 同时生成 harmful 与 benign counterpart，防止 over-refusal。
10. 将不同 hypothesis 分成 train / validation / OOD。
11. 按当前 failure rate 动态上采样最薄弱的 hypothesis。
12. 修复后同时检查 safety failure、false positive 与 capability tax。

这与 A3 的公开流程高度一致。[9]

12.3 Long-horizon Agent：Outcome failure $\rightarrow$ Step-level causal record

13. 把 trajectory 标准化成 state/action/observation。
14. 提取每一步约束与期望状态。
15. 定位 first critical failure step。
16. 建立 error propagation DAG，区分 root cause 与 downstream symptom。
17. 若是 tool/harness 故障则路由系统修复；若是 policy 决策错误再进入训练。
18. 修复后运行相同 seed + 近邻变体，确认不是记忆单例。

13. Production Failure Mining Architecture



服务	职责	关键输出
Signal Ingest	Eval / prod / user / red-team 数据接入	normalized failure candidates
Failure Validator	检查 spec / grader / env / reproducibility	failure confidence
Trace Analyzer	step-level / DAG / first divergence	root cause
Taxonomy & Cluster	多标签分类 + 聚类	failure family
Priority Scheduler	impact / frequency / learnability / cost	gap queue
Data Compiler	SFT / pair / rollout / verifier 数据生成	train-ready records
Regression Builder	构造 private/OOD 回归集	release gates
Feedback Controller	追踪修复效果与新回归	closed-loop state

13.1 推荐数据契约

```
failure_id: fm_2026_001284
source: production | eval | redteam | rl_rollout
model_version: ...
task_id: ...
trajectory_uri: ...
observed_outcome: fail
validation:
  spec_valid: true
  grader_valid: true
  reproducible_rate: 0.83
  diagnosis_confidence: 0.91
root_cause:
  taxonomy: [tool.argument, recovery.no_retry]
  critical_step: 7
  parent_failure_id: null
priority:
  severity: high
  frequency: medium
  learnability: high
  novelty: 0.72
route:
  primary: agent_sft
  secondary: private_regression
fix_status: queued
```

14. Failure Mining 的核心指标

指标	解释
Validated Failure Rate	候选失败中有多少是真 failure
Root-Cause Accuracy	关键失败步骤/类别与专家一致度
Cluster Coverage	主要 failure 是否进入稳定 taxonomy
New Failure Yield	每单位分析预算发现的新 failure family
Near-Miss Yield	进入 learnable frontier 的样本比例
Fix Precision	定向修复是否真的降低目标 failure
OOD Fix Generalization	未见近邻/新场景是否也改善
Regression Cost	修复造成其他能力退化多少
Time-to-Diagnosis	从发现到 root cause 的时间
Time-to-Trainable-Signal	从 failure 到可训练数据的时间
Recurrence Rate	已修 failure 在未来版本复发率
Evaluator Error Rate	failure mining 中被发现的 eval bug 比例

不要只优化 “bad cases fixed”。真正重要的是：修复是否跨 paraphrase、跨 domain、跨 model version 泛化，以及是否造成 collateral regression。

15. 18 类常见 Failure Mining 失败模式

#	Failure Mining 反模式	后果
1	把 Eval Bug 当模型 Bug	错误标签进入训练
2	只收最终失败，不看 trajectory	无法定位 root cause
3	只按语义聚类	不同因果机制被混合
4	taxonomy 太细	统计稀疏、难维护
5	taxonomy 太粗	无法指导干预
6	只收 Always-Fail	不可学习、浪费预算
7	忽略 Near-Miss	丢失最大梯度区
8	把传播错误重复计数	同一 root cause 被多次惩罚
9	只做 offline failure mining	看不到真实分布 drift
10	只依赖生产投诉	罕见高影响 failure 被漏掉
11	Judge 未校准	失败/修复判断共同偏移
12	Failure → SFT 一刀切	本应修 tool/eval/product
13	只做 harmful examples	导致 over-refusal / false positive
14	只修原 prompt	学会 memorization 而非机制
15	没有 OOD split	修复看似成功但不泛化
16	不记录 lineage	无法解释哪次修复导致回归
17	修复后不进 regression	同类问题反复出现
18	只看 failure count	忽略 severity × impact × cost

16. Failure Mining 成熟度模型

等级	系统形态	主要能力
L0 - Logs	人工查看 bad cases	没有 taxonomy / route
L1 - Taxonomy	稳定错误分类 + regression	能统计失败分布
L2 - Root Cause	step/trajectory 分析	能定位 critical step
L3 - Data Flywheel	failure 自动编译成训练数据	SFT/DPO/RL route
L4 - Adaptive Controller	主动发现 + 优先级 + 自动 OOD 验证	闭环自适应后训练

16.1 一个可执行的最小实现（MVP）

- 19. 建立 8-15 个一级 failure taxonomy，禁止一开始追求上百类别。
- 20. 每个 eval failure 强制保存原始 input/output/judge/trace。
- 21. 先实现 failure validator：任务规格、grader、环境、可复现性四项检查。
- 22. 每周抽样失败，生成 top failure clusters 与 cross-slice 报告。
- 23. 每类 cluster 明确 primary intervention route。
- 24. 每个已修 cluster 自动进入 private regression；至少加入 3-10 个语义近邻。
- 25. 按 $\text{impact} \times \text{frequency} \times \text{learnability}$ 建 Gap Queue，驱动下一轮 Prompt Factory/Data Factory。

17. 最终 Know-Why / Know-How

原则	Know-Why	Know-How
Failure 先验证	错误监督比缺失监督更危险	先查 task/grader/env/harness
看轨迹而非只看结果	长程错误会传播	找 first divergence + causal propagation
找 family 而非单例	泛化来自机制级修复	cluster + slice + hypothesis
Near-Miss 优先	最有学习梯度	按 pass-rate / margin 找 frontier
路由比造数据重要	不是每个 bug 都该训模型	Prompt/SFT/Pref/RL/Eval/Harness router
修复后必须 OOD	防 memorization	近邻变体 + disjoint hypothesis
保留真实锚点	synthetic auditor 会有偏差	production logs + expert calibration
Failure 资产长期累计	模型会升级，问题会复发/变形	lineage + regression + versioning

最终心智模型：Evaluation 是传感器；Failure Mining 是诊断器；Post-Training 是执行器；Regression / Production Monitoring 是闭环反馈。

18. 核心资料与推荐阅读

[1] OpenAI (2025). How evals drive the next chapter in AI for businesses.

<https://openai.com/index/evals-drive-next-chapter-of-ai/>

[2] Anthropic (2026). Demystifying evals for AI agents.

<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

[3] OpenAI (2026). Separating signal from noise in coding evaluations.

<https://openai.com/index/separating-signal-from-noise-coding-evaluations/>

[4] Barke et al. (2026). AgentRx: Diagnosing AI Agent Failures from Execution Trajectories.

<https://arxiv.org/abs/2602.02475>

[5] Guo et al. (2026). AgentEval: DAG-Structured Step-Level Evaluation for Agentic Workflows.

<https://arxiv.org/abs/2604.23581>

[6] Anthropic (2025). Petri: An open-source auditing tool to accelerate AI safety research.

<https://www.anthropic.com/research/petri-open-source-auditing>

[7] Anthropic Alignment Science (2026). Petri 2.0.

<https://alignment.anthropic.com/2026/petri-v2/>

[8] Anthropic Alignment Science (2025). Bloom: automated behavioral evaluations.

<https://alignment.anthropic.com/2025/bloom-auto-evals/>

[9] Anthropic Fellows (2026). A3: An Automated Alignment Agent for Safety Finetuning.

<https://alignment.anthropic.com/2026/automated-alignment-agent/>

[10] OpenAI (2026). GPT-Red: Unlocking Self-Improvement for Robustness.

<https://openai.com/index/unlocking-self-improvement-gpt-red/>

[11] Kumar et al. (2024). Training Language Models to Self-Correct via Reinforcement Learning (SCoRe).

<https://arxiv.org/abs/2409.12917>

[12] Gao et al. (2026). How to Interpret Agent Behavior (ACT*ONOMY).

<https://arxiv.org/abs/2605.13625>

[13] Zhan et al. (2026). Why Your Deep Research Agent Fails?

<https://arxiv.org/abs/2601.22984>

[14] OpenAI (2026). Predicting model behavior before release by simulating deployment.

<https://openai.com/index/deployment-simulation/>

[15] Marks et al. (2025). Auditing language models for hidden objectives.

<https://arxiv.org/abs/2503.10965>

[16] Richter et al. (2024). An Auditing Test To Detect Behavioral Shift in Language Models.

<https://arxiv.org/abs/2410.19406>

建议阅读顺序

26. 先读 OpenAI 的 Specify → Measure → Improve 与 Anthropic Agent Evals，建立“评测必须导向错误分析”的工程观。
27. 再读 AgentRx / AgentEval，理解 trajectory-level root cause。
28. 读 Petri → Bloom，理解“发现未知 failure”与“测量已知 failure”的区别。
29. 读 A3，理解单个 failure 如何扩成 hypothesis、train/val/OOD 与 adaptive reweighting。
30. 最后读 GPT-Red，理解 failure discovery 如何升级成对抗 self-play 与自我改进飞轮。